



I have given the app name as 'flow'.

Task:

1. Login & Registration (with validation)
2. Home Screen (API integration: <https://jsonplaceholder.typicode.com/posts>)
3. Profile Screen (display stored user data)
4. Local storage using AsyncStorage

Tech Stack:

1. **Core Technologies :**
 - React Native
 - JavaScript
2. **Navigation:**
 - React Navigation
3. **State Management:**
 - Redux Toolkit
 - Redux Persist
4. **Api Handling:**
 - Axios
5. **Forms & Validation**
 - Formik
 - yup
6. **Storage**
 - AsyncStorage
7. **UI & Utilities :**
 - React Native Paper
 - FlashList
 - Vector Icons

Features Implemented:

1. Authentication

Registration Screen:

- User registration using local storage
- Duplicate email validation (check email from storage and raises error if email already in use)
- Password validation rules
 - 8+ characters
 - uppercase letter
 - lowercase letter
 - number
 - special character
- Login page redirect after successful registration

Login Screen:

- Form validation using Formik + Yup
- Email validation
- Strong password validation
- Password visibility toggle (Used eye toggle)
- Snackbar messages (Incorrect Password/ User doesn't exist/ Logged in/ Registered)
- Keyboard handling (Close keyboard by clicking outside it, move form above the keyboard)
- Added App Icon for UI

Authentication Flow:

Registration -> Login

The application validates:

- Whether the user exists
- Whether the password is correct

2. Home Screen

API Integration:

<https://jsonplaceholder.typicode.com/posts>

Posts List Rendering:

FlashList Implementation:

Instead of FlatList, FlashList was used for:

- Better performance
- Optimized rendering
- Reduced memory usage
- Smooth scrolling experience

Pagination:

Instead of rendering all posts together:

- Initially only 10 posts are rendered
- Additional posts load during scrolling
- Infinite scrolling behavior implemented

Search Functionality:

Implemented real-time post searching using:

- Title filtering
- Case-insensitive search

3. Profile Screen

The Profile Screen displays:

- User name
- User email
- Stored authentication details

Data is persisted locally using Redux Persist.

Project Structure:

Followed a scalable folder structure

Src

- Api
 - Posts.js
- Components
 - Common
 - AppButton
 - AppImage
 - AppInput
 - AppSnackBar
 - AppText
 - ScreenWrapper
 - cards
- Navigation
 - AppNavigator
 - BottomTabs
- Redux
 - Store.js

- Slices
 - AuthSlice.js
- Screens
 - Auth
 - LoginScreen.js
 - RegisterScreen.js
 - Home
 - HomeScreen.js
 - Profile
 - ProfileScreen.js
- Theme
 - Colors.js
 - Spacing.js
 - Typography.js
- Utils
 - ValidationSchema.js